# Python Final Project

## 1. Database & Data Processing (Option A)

### 1.1. Database Setup

- **customers.csv** : customer_id, name, email, city, join_date
- **orders.csv** : order_id, customer_id, product_id, quantity, order_date
- **products.csv** : product_id, name, category, price, cost

i. Creates a new SQLite database file

```python
def create_database(db_path: str) -> None:
    """Create SQLite database and tables for the e-commerce dataset."""
    db_file = Path(db_path)
    db_file.parent.mkdir(parents=True, exist_ok=True)

    with sqlite3.connect(db_path) as conn:
        cursor = conn.cursor()

         # Enable foreign key support: Activer les clés étrangères
        cursor.execute("PRAGMA foreign_keys = ON;")

        # (Optionnel) nettoyer les tables pour pouvoir relancer le script
        # Drop tables if they exist (for easier re-runs during development)
        cursor.execute("DROP TABLE IF EXISTS orders;")
        cursor.execute("DROP TABLE IF EXISTS products;")
        cursor.execute("DROP TABLE IF EXISTS customers;")
```

The SQLite database file (myproject.db) is created when sqlite3.connect(db_path) is executed in the create_database() function. If the file does not exist, SQLite automatically creates it. The script also ensures the folder exists using db_file.parent.mkdir(parents=True, exist_ok=True).

ii. Defines table schemas

I implemented the database schema in the create_database() function. Each table has a primary key (customer_id, product_id, order_id), and the orders table includes foreign keys referencing customers(customer_id) and products(product_id). I used appropriate data types (INTEGER, TEXT, REAL) and constraints such as NOT NULL and UNIQUE on the email field to enforce data integrity. Foreign key support is explicitly enabled with PRAGMA foreign_keys = ON;.

```python
# === TABLE customers ===
# Primary key: customer_id
# Constraints: email UNIQUE, name and join_date NOT NULL
cursor.execute(
    """
    CREATE TABLE IF NOT EXISTS customers (
        customer_id INTEGER PRIMARY KEY,
        name        TEXT NOT NULL,
        email       TEXT NOT NULL UNIQUE,
        city        TEXT,
        join_date   TEXT NOT NULL
    );
    """
)
```

```python
# === TABLE products ===
# Primary key: product_id
# Constraints: price and cost NOT NULL
cursor.execute(
    """
    CREATE TABLE IF NOT EXISTS products (
        product_id INTEGER PRIMARY KEY,
        name       TEXT NOT NULL,
        category   TEXT,
        price      REAL NOT NULL,
        cost       REAL NOT NULL
    );
    """
)
```

```python
# === TABLE orders ===
# Primary key: order_id
# Foreign keys: customer_id -> customers, product_id -> products
# Constraints: quantity and order_date NOT NULL
cursor.execute(
    """
    CREATE TABLE IF NOT EXISTS orders (
        order_id    INTEGER PRIMARY KEY,
        customer_id INTEGER NOT NULL,
        product_id  INTEGER NOT NULL,
        quantity    INTEGER NOT NULL,
        order_date  TEXT NOT NULL,
        FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
        FOREIGN KEY (product_id)  REFERENCES products(product_id)
    );
    """
)
```

iii. Imports data from CSV files into the database

```python
def import_csv(db_path: str, csv_file: str, table_name: str, skip_header: bool = True) -> None:
    """
    Import a CSV file into a database table.

    Assumes:
    - The first row of the CSV is a header row with column names
      matching the table's column names.
    """
    csv_path = Path(csv_file)
    if not csv_path.exists():
        raise FileNotFoundError(f"CSV file not found: {csv_file}")

    with sqlite3.connect(db_path) as conn, open(csv_path, newline="", encoding="utf-8") as f:
        cursor = conn.cursor()
        reader = csv.reader(f)

        # Lire l'en-tête
        # Read header row to build dynamic INSERT statement
        header = next(reader) if skip_header else None
        if header is None:
            raise ValueError("CSV must have a header row for dynamic insert.")

        columns = ", ".join(header)
        placeholders = ", ".join(["?"] * len(header))
        insert_sql = f"INSERT INTO {table_name} ({columns}) VALUES ({placeholders})"

        rows = list(reader)
        cursor.executemany(insert_sql, rows)
        conn.commit()

    print(f"Imported {len(rows)} rows from {csv_file} into {table_name}")
```

iv. Verifies successful import (print row counts)

After importing the CSV data, I call the verify_data() function, which connects to the SQLite database and runs SELECT COUNT(*) on each table (customers, products, orders). It prints the row counts to the console, allowing me to confirm that all records from the CSV files were successfully loaded into the database.

```
Database created: data/myproject.db
Imported 30 rows from data/customers.csv into customers
Imported 20 rows from data/products.csv into products
Imported 100 rows from data/orders.csv into orders

=== Row counts ===
Table customers: 30 rows
Table products: 20 rows
Table orders: 100 rows
```

## 1.2. Data Analysis Queries

In Task 1.2, I developed five analytical queries designed to explore and extract key insights from the database created in the previous task. The objective was to combine SQL-based data retrieval with Python-based processing to demonstrate a complete understanding of how a data-driven application operates. The first query illustrates a simple filtering operation using a WHERE clause, while the second introduces a join and aggregation to analyze revenue by product category. Queries 3, 4, and 5 were implemented in two versions: an SQL version using JOIN, GROUP BY, and aggregation functions, and a Python version that replicates the same logic using loops, dictionaries, and sorting operations. This dual approach demonstrates the ability to perform analytical computations both at the database level and within the application layer, making it possible to generate business-relevant insights such as total customer spending, city-level sales performance, and product profitability.

**Query 1 — Customers by City**

**Business Question:**

*Which customers live in a specific city, enabling targeted local marketing or logistics planning?*

This query retrieves customers filtered by city using a simple WHERE clause. It illustrates a basic segmentation operation commonly used in customer analytics.

**Query 2 — Revenue by Product Category**

**Business Question:**

*Which product categories generate the highest revenue for the company?*

This query joins the orders and products tables, computes revenue per order line (quantity × price), and aggregates the results by product category. It identifies top-performing categories and helps inform product and marketing strategy.

**Query 3 — Top Customers by Total Spending (SQL + Python)**

**Business Question:**

*Who are the company's highest-value customers based on total spending, and how could they be targeted with loyalty initiatives?*

This analysis calculates total spending per customer. The SQL version uses SUM and GROUP BY, while the Python version manually rebuilds the aggregation from raw query results. It highlights which customers contribute the most to overall revenue.

**Query 4 — Sales Summary by City (SQL + Python)**

**Business Question:**

*Which cities generate the highest sales, and where are the most active markets located?*

This query computes total sales revenue and the number of orders per city. It requires joining customers, orders, and products. The Python version replicates the aggregation logic using dictionaries. The analysis provides geographical performance insights crucial for market planning.

**Query 5 — Product Profitability (SQL + Python)**

**Business Question:**

*Which products contribute the most to overall profit, and which should be prioritized in the commercial strategy?*

This query calculates product-level profitability using: profit = SUM(quantity × (price – cost))

The Python version performs the same computation by iterating over all order rows. The resulting ranking helps identify high-margin products and distinguish them from less profitable items.

```
=== Analysis Results ===

1. Customers in Boston:
(1, 'Sarah Johnson', 'sarah.j@email.com', 'Boston', '2023-01-15')
(3, 'Emma Davis', 'emma.d@email.com', 'Boston', '2023-03-10')
(5, 'Olivia Brown', 'olivia.b@email.com', 'Boston', '2023-04-05')
(8, 'Benjamin Lee', 'ben.lee@email.com', 'Boston', '2023-05-15')
(11, 'Isabella Thomas', 'i.thomas@email.com', 'Boston', '2023-06-28')
(14, 'Daniel White', 'd.white@email.com', 'Boston', '2023-08-05')
(17, 'Amelia Lewis', 'amelia.l@email.com', 'Boston', '2023-09-15')
(20, 'David Allen', 'd.allen@email.com', 'Boston', '2023-10-25')
(22, 'Jackson King', 'jack.k@email.com', 'Boston', '2023-11-20')
(25, 'Emily Scott', 'emily.s@email.com', 'Boston', '2024-02-05')
(27, 'Sofia Adams', 'sofia.a@email.com', 'Boston', '2024-04-01')
(30, 'Joshua Carter', 'josh.c@email.com', 'Boston', '2024-07-15')

2. Revenue by product category (SQL):
('Electronics', 3506.06)
('Accessories', 748.67)
('Office Supplies', 396.65999999999997)

3. Top customers by total spending (SQL):
(6, 'William Martinez', 249.94)
(3, 'Emma Davis', 235.89999999999998)
(25, 'Emily Scott', 231.94)
(9, 'Ava Anderson', 215.93)
(2, 'Michael Chen', 203.95999999999998)

3b. Top customers by total spending (Python):
(6, 'William Martinez', 249.93999999999997)
(3, 'Emma Davis', 235.89999999999998)
(25, 'Emily Scott', 231.94)
(9, 'Ava Anderson', 215.92999999999998)
(2, 'Michael Chen', 203.95999999999998)
```

```
4. City-level sales summary (SQL):
('Boston', 1927.29, 44)
('New York', 1139.67, 22)
('Chicago', 972.67, 19)
('San Francisco', 611.76, 15)

4b. City-level sales summary (Python):
('Boston', 1927.2900000000004, 44)
('New York', 1139.67, 22)
('Chicago', 972.6700000000002, 19)
('San Francisco', 611.7600000000001, 15)

5. Product profitability (SQL):
(120, 'Headphones', 'Electronics', 719.92, 360.0, 359.91999999999996)
(106, 'Ergonomic Keyboard', 'Electronics', 719.91, 360.0, 359.90999999999997)
(101, 'Wireless Mouse', 'Electronics', 689.77, 345.0, 344.77)
(102, 'USB-C Cable', 'Electronics', 376.71, 145.0, 231.71)
(118, 'Webcam', 'Electronics', 349.95, 175.0, 174.95)
(107, 'Monitor Stand', 'Electronics', 335.94, 168.0, 167.94)
(111, 'Water Bottle', 'Accessories', 279.86, 112.0, 167.85999999999999)
(110, 'Laptop Backpack', 'Accessories', 299.94, 150.0, 149.94)
(115, 'Coffee Mug', 'Accessories', 168.87, 65.0, 103.87)
(112, 'Desk Lamp', 'Electronics', 209.94, 108.0, 101.94000000000001)

5b. Product profitability (Python):
(120, 'Headphones', 'Electronics', 719.92, 360.0, 359.91999999999996)
(106, 'Ergonomic Keyboard', 'Electronics', 719.91, 360.0, 359.90999999999997)
(101, 'Wireless Mouse', 'Electronics', 689.7700000000001, 345.0, 344.7700000000001)
(102, 'USB-C Cable', 'Electronics', 376.71, 145.0, 231.70999999999998)
(118, 'Webcam', 'Electronics', 349.95, 175.0, 174.95)
(107, 'Monitor Stand', 'Electronics', 335.94, 168.0, 167.94)
(111, 'Water Bottle', 'Accessories', 279.85999999999996, 112.0, 167.85999999999996)
(110, 'Laptop Backpack', 'Accessories', 299.94, 150.0, 149.94)
(115, 'Coffee Mug', 'Accessories', 168.87, 65.0, 103.87)
(112, 'Desk Lamp', 'Electronics', 209.94000000000003, 108.0, 101.94000000000003)
```

### 1.3. Data Export & Reporting

For Task 1.3, I selected a query that identifies the top products based on sales volume. The query joins the orders and products tables, then calculates, for each product, the total quantity ordered and the total revenue (SUM(quantity × price)). The results are grouped by product and sorted in descending order of quantity sold to highlight the best-selling items. These aggregated results are then exported to a CSV file (top_products.csv) using Python's csv module, allowing the data to be reused in external tools such as Excel or Tableau for additional analysis or reporting.

```
CSV exported: top_products.csv
Text report generated: report_task1-3.txt
```

## 2. API Demonstration - Simple API call

For Part 2, I used the REST Countries API, which does not require an API key and returns structured JSON data. I designed an interactive script that prompts the user to enter a country name, sends a request to https://restcountries.com/v3.1/name/{country}, and then extracts key information such as the official country name, capital city, population, region, and main languages. The script uses Python's built-in urllib.request and json modules, and includes basic error handling (for example, when the country is not found or the response structure is different than expected). This simple example demonstrates my ability to call a public API, parse JSON responses, and present the results in a readable format.

```
=== API Demonstration: REST Countries ===

Enter a country name (e.g., France, Japan, Brazil):  France

--- Country Information ---
Official Name : French Republic
Capital       : Paris
Population    : 66351959
Region        : Europe
Languages     : French

API demonstration complete!
```

### Optional: api_integration

As an optional extension, I implemented an API integration that combines a database query with an external API call. The script retrieves a city directly from the SQLite database (from the customers table) and uses it as input to call the Open-Meteo API, which does not require an API key. The API returns current weather information for the selected city, such as temperature and wind speed, which is then displayed to the user. This optional component demonstrates how database content can be used dynamically as input for an external API, illustrating an effective integration between persistent data storage and live external data sources.

```
=== Optional API Integration ===

City from DB: Boston
Current weather in Boston: 0.4°C, wind 7.7 km/h

Optional API integration complete!
```

# 3. Reflection - AI usage and learning summary

## 3.1. Section 1: Project Overview

*Which option did I choose for Part 1? Why?*

I chose **Option A**, which uses the instructor-provided e-commerce dataset (customers.csv, products.csv, orders.csv).

I selected this option because the data is clean, well-structured, and optimized for the project requirements. It allowed me to focus on designing the database schema, implementing SQL queries, and developing Python-based data processing logic, rather than spending additional time cleaning or restructuring my own midterm CSV files. Option A also ensured that all tasks—especially the SQL/Python dual versions of the analytical queries—could be completed efficiently and with fewer data-quality issues.

*Which API(s) did I use in Part 2?*

For Part 2, I used the **REST Countries API**, which does not require an API key and returns structured JSON data.

My script prompts the user to enter a country name and then retrieves basic information such as capital city, population, region, and languages. I chose this API because it is simple, reliable, and easy to test, making it ideal for demonstrating API calling, JSON parsing, and error handling.

*Brief summary of what the project does*

My project creates a small, end-to-end Python application that integrates **database creation**, **data loading**, **SQL queries**, **Python-based data analysis**, **file exports**, and a **live API call**.

- In **Part 1**, I built an SQLite database from the provided CSV files, created normalized table schemas with foreign keys, and imported all records. I then implemented five analytical queries, including SQL and Python versions for the aggregation tasks, to answer business questions such as customer spending, revenue by product category, and monthly sales trends.
- In **Part 1.3**, I generated both a CSV export and a text-based summary report.
- In **Part 2**, I developed an interactive API script that fetches real-time information from the REST Countries API based on user input.
- Overall, the project demonstrates my ability to combine database management, data processing, file I/O, and basic API integration in Python, following best practices and robust error handling.

## 3.2. Section 2: Technical Challenges & Solutions

The most complex challenge was **implementing SQL and Python versions of grouped analysis queries (Queries 3-5)**. Specifically:

- **Problem**: Ensuring that the results from SQL queries (using GROUP BY, SUM, etc.) and Python versions (using loops and dictionaries) were **identical**. For example, for the "top customers by total spending" query, the Python version had to manually replicate the grouping and sorting logic.
- **Solution**:
  - I first wrote the SQL query to get the expected result, then broke down the logic into Python steps:
    1. Fetch raw data with a simple SELECT query (no GROUP BY).
    2. Use a dictionary to aggregate amounts by customer ({customer_id: total_spent}).
    3. Sort the dictionary by value in descending order and return the top N results.
  - I validated the results by comparing the outputs of both versions and ensuring they matched exactly
- **What I Would Do Differently**:
  - Use **pandas** for the Python versions of grouped queries, which would simplify data manipulation (e.g., df.groupby()). However, since the project encouraged a "basic" approach, I stuck to native Python structures to better understand the underlying logic.

## 3.3. Section 3: AI Usage Documentation

Here are concrete examples of how I used AI tools (like **Le Chat** or **GitHub Copilot**) during the project.

*Example 1: Database Schema Creation (Helpful)*

- **Prompt**: *"Generate an SQLite schema for an e-commerce database with 3 tables: customers, products, and orders. Include primary keys, foreign keys, and NOT NULL constraints."*
- **What AI Suggested**: A complete SQL script with tables and relationships, including examples of constraints.

```sql
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT UNIQUE,
    city TEXT
);
```

- **Validation/Modification**:
    - I verified that the data types matched the CSV files (e.g., order_date as TEXT for SQLite).
    - I added comments to explain each table and included PRAGMA foreign_keys = ON; to enable foreign keys.
- **Evaluation**: **Very helpful** for saving time on SQL syntax, but I had to adapt details to match my data.

*Example 2: SQL Query with JOIN (Partially Helpful)*

- **Prompt**: *"Write a SQL query to calculate revenue by product category, joining the orders and products tables, and grouping by category."*
- **What AI Suggested**:

```sql
SELECT p.category, SUM(o.quantity * o.unit_price)
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY p.category;
```

- **Problem**: The query assumed a different column structure than my actual schema, requiring me to adjust column references and verify the business logic against my real data
- **Validation/Modification**:

    - I corrected the join and adjusted column references to match my actual schema, then verified the business logic by testing the query results in DB Browser for SQLite.
    - I tested the query in DB Browser for SQLite to validate the results.
- **Evaluation**: **Partially helpful**—AI provided a good structure, but I had to fix the business logic details.

*Example 3: Error Handling for API (Helpful)*

- **Prompt**: *"How to handle errors for an API request in Python using urllib? For example, if the user enters an invalid country name."*
- **What AI Suggested**:
    - Use a try/except block to catch urllib.error.URLError (network error) and KeyError (missing field in JSON response).

```python
try:
    with urllib.request.urlopen(url) as response:
        data = json.loads(response.read().decode())
        return data
except urllib.error.URLError as e:
    print(f"Network error: {e}")
    return None
```

- **Validation/ Modification**:
    - I added a check for HTTP 404 responses (country not found) by analyzing the status code.
    - I also improved the user message: *"Country not found. Please check the spelling."*
- **Evaluation**: **Very helpful**—the base code was solid, and I enhanced it for better user experience.

*Example 4: Data Export to CSV (Misleading)*

- **Prompt**: *"How to export SQLite query results to a CSV file in Python?"*
- **What AI Suggested**:
    - Use pandas.DataFrame.to_csv(), which is efficient but requires installing pandas.
- **Problem**: I deliberately avoided external libraries to keep the solution lightweight and aligned with project expectations.
- **Validation/Modification**:
    - I implemented a solution using the csv module:

```python
with open('top_products.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['product_id', 'name', 'total_sales'])
    writer.writerows(results)
```

- **Evaluation**: **Misleading** in this context, as the suggestion relied on an external library. However, it reminded me to clarify technical constraints before using AI.

*Example 5: Optimizing Python Queries (Insightful)*

- **Prompt**: *"How to optimize a Python loop that aggregates data from a list of tuples?"*
- **What AI Suggested**:
    - Use defaultdict to simplify aggregation

```python
from collections import defaultdict
sales_by_category = defaultdict(int)
for row in data:
    sales_by_category[row['category']] += row['amount']
```

- **Validation/Modification:**

I tested this approach and compared it to my initial version with a standard dictionary. The results were identical, but defaultdict made the code more readable.

- **Evaluation**: Very helpful—this suggestion taught me a better practice for aggregations in Python.

## 3.4. Section 4: Learning Reflection

- **Python + Databases**:
    - I learned how **sqlite3** allows me to manipulate a database entirely in Python, without a GUI like DBeaver. This is particularly useful for automating tasks (e.g., data imports/exports).
    - The main difference from DBeaver: In Python, you must manually manage connections, transactions, and errors, which makes the code more verbose but also more flexible for automated scripts.
- **When to Use Python vs. SQL GUI**:
    - **SQL GUI (DBeaver, SQLite Browser)**: Ideal for exploring data, quickly testing queries, or visualizing a schema.
    - **Python**: Essential for automating processes (e.g., importing CSVs, generating reports), integrating data with APIs, or performing post-query processing (e.g., complex calculations, file exports).
- **Python Skills to Develop Further**:
    - **Pandas**: To manipulate tabular data more efficiently than with loops and dictionaries.

    - **Unit Testing**: Writing tests to validate query functions and export logic.
    - **Asynchronous Programming**: Understanding how to make asynchronous API calls to improve performance.

- **Reflection on AI Usage**

AI was a **time-saver** for repetitive tasks (e.g., SQL syntax, error handling), but I learned to:

- **Never copy-paste without understanding**: Several suggestions contained subtle errors (e.g., incorrect column names, incompatible SQL functions).
- **Validate with tests**: I systematically tested queries and Python functions with small datasets before integrating them.
- **Document limitations**: In my reflection, I noted cases where AI was misleading (e.g., pandas dependency), which helped me refine my prompts.